



Qualcomm Technologies, Inc.

Qualcomm Linux Video Guide

80-70014-20 Rev. AH

July 12, 2024

Contents

- 1 Overview 5
- 2 Getting started 7
- 3 Features 9
- 4 Architecture 16
- 5 Interfaces 18
- 6 Software 26
- 7 Verify 29
- 8 Configure 32
- 9 Debug 39
- 10 References 42
 - 10.1 Related documents 42
 - 10.2 Acronyms and terms 42

Figures

Figure 1-1: High-level architecture of the video software..... 6

Figure 3-1: Hier-P layer encoding pattern..... 10

Figure 3-2: Long Term Reference encoding.....12

Figure 4-1: Video decoder software architecture..... 16

Figure 4-2: Video encoder software architecture..... 17

Figure 9-1: Debug VPU workflow.....39

Tables

Table 3-1: Features applicable to QCS6490.....9

Table 3-2: Adreno VPU decoder capabilities..... 13

Table 3-3: Adreno VPU encoder capabilities..... 14

Table 3-4: Adreno VPU feature description..... 14

1 Overview

The Qualcomm® Adreno™ Video Processing Unit (VPU) supports the hardware-based video decoding and encoding for various video codecs. The video encoding and decoding in Qualcomm-based SDKs are offloaded to the video engines to ensure that the CPU and GPU are free for other operations.

The video features of the Adreno VPU engines are exposed through the Qualcomm Linux Intelligent Multimedia Software Development Kit (QIM SDK) framework and the V4L2 (Video for Linux, version 2) APIs. For more information, see [Qualcomm Linux Intelligent Multimedia Software Development Kit \(QIM SDK\) Reference](#).

The following figure shows the high-level architecture of the video software. Applications based on the V4L2 or GStreamer APIs can offload the encoding or decoding to Adreno VPU using the V4L2 API framework.

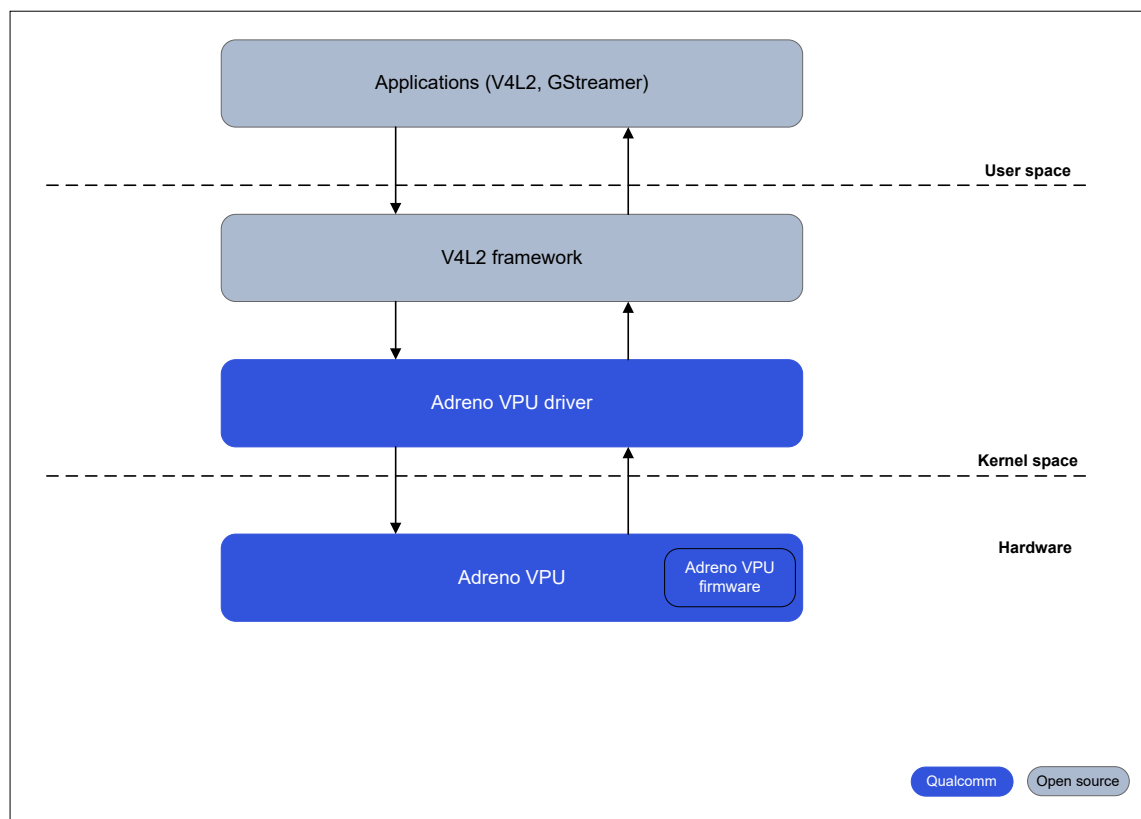
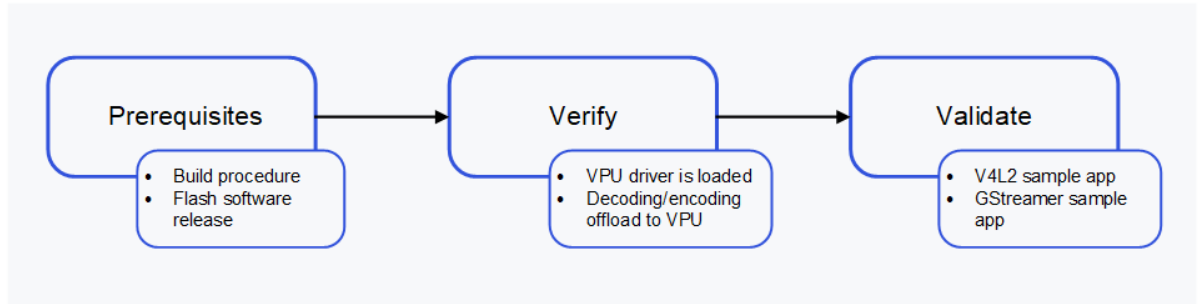


Figure 1-1 High-level architecture of the video software

NOTE The Qualcomm Linux platform allows you to develop applications for QCS6490 and QCS5430.

2 Getting started



Prerequisites

- Set up your infrastructure as described in the [Qualcomm Linux Build Guide](#).
- [Flash the latest software release](#) to the RB3 Gen2 development board.
- Set up SSH connection:
 - a. Enable SSH in Permissive mode by performing the steps mentioned in [How to SSH](#).
 - b. Connect to the device by running the following command:

```
ssh root@<device_IP_address>
```

For example, if the IP address of the device is 10.92.160.222, run the following command:

```
ssh root@10.92.160.222
```

Verify

To verify if the VPU driver is loaded properly and whether the decoding and encoding is offloaded to VPU, see [Verify](#).

Validate

The video functionality can be effectively used either at the native layer by using the V4L2 interfaces, or by using the QIM SDK framework. Qualcomm provides V4L2 and GStreamer-based sample applications that enable you to run the sample use cases.

V4L2 samples

The `iris_v4l2_test` is a simple video test application that uses Linux V4L2 interfaces to simulate several basic encoder and decoder behaviors. For more information, see [sample V4L2 application](#).

GStreamer samples

The sample GStreamer applications for video playback are part of the QIM SDK. To understand more about the prerequisites for the GStreamer applications, see [Sample applications](#).

The following table lists the methods and sample video test cases:

Method	Use case
Using <code>gst-launch-1.0</code>	Video only playback
	Audio and video playback
	Video recording
Using GST application	Video playback and recording

3 Features

The Adreno VPU provides various video decoding and encoding features. The following table lists the features applicable to this platform:

Table 3-1 Features applicable to QCS6490

Use case	Feature	QCS6490
Codecs	H.264 (8-bit decoder)	Yes
	HEVC (8-bit decoder)	Yes
	HEVC (10-bit decoder)	Yes
	VP9 (8-bit decoder)	Yes
	VP9 (10-bit decoder)	Yes
	H.264 (8-bit encoder)	Yes
	HEVC (8-bit encoder)	Yes
Features	B-frame encode	Yes
	Encoder initial QP override	Yes
	Hierarchical-P encode	Yes
	Slice encode	Yes
	Intra-refresh	Yes
	Preprocessing	Yes
	Rate control	Yes
	Long term reference (LTR)	Yes
	Dynamic properties	Yes
Concurrency	Multi-instance support	Yes

Feature descriptions

The supported decode and encode features are described here.

B-frame encode

B-Frame is a type of frame that uses both earlier and future frames as data reference to get the highest amount of data compression. The Adreno VPU can encode frames with adaptive B type to get the highest possible compression without compromising on the video quality.

Encoder initial QP override

Video encoding involves mapping signal levels to discrete values that are easily compressed. Quantization is a lossy process, and the levels of quantization govern the quality versus compression.

Encoders typically start with a default Quantization Parameter (QP) at the beginning. Based on the configured bit rate and scene complexity, encoders try to arrive at the right QP value by continuously monitoring the complexity and redundancy across frames. It may take a few seconds for the encoder to reach a steady state and predict the right QP value that matches the target bit rate (also known as rate convergence).

Hierarchical-P encode

With the Hierarchical-P (Hier-P) feature, the encoder organizes the frames into multiple layers, with frames of one layer referencing only frames from the lower layers as shown in the following figure. The lowest layer, also known as layer 0 or the base layer, is the only exception.

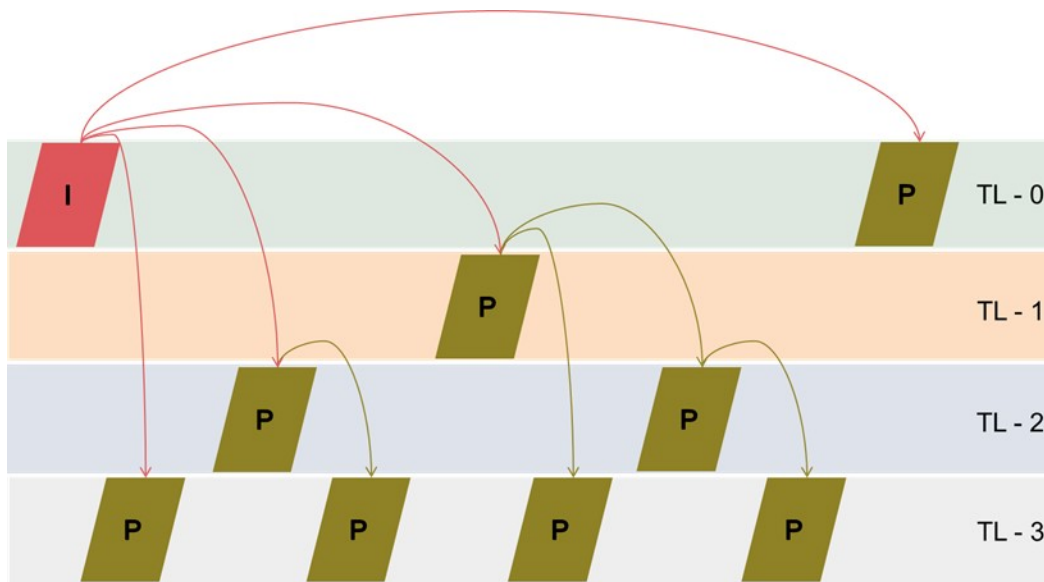


Figure 3-1 Hier-P layer encoding pattern

In the preceding figure, the TL-0 represents the base layer, and the remaining layers represent the enhancement layers. Hier-P improves error resilience and temporal scalability. The Hier-P feature is useful for video telephony (VT) or video conferencing applications that involve channel errors. Hier-P allows you to control error propagation by selectively dropping the enhancement layers.

Slice encode

Encoders can compress a frame with an independently decodable Group-of-Blocks (GOBs), also known as slices. Each slice is independently decodable, and are intended to be units of recovery if there is a data loss or corruption. The following are the advantages of introducing slices in an encoded frame:

- A corrupt slice can be ignored and skipped to a next slice, thus restricting the corruption to a part of the frame instead of the entire frame.
- Slices can be sized to fit within a network packet to help with transmission.
- Erroneous slices can be retransmitted instead of sending the whole frame.
- Applications can use slices to reduce latency in real-time communication. Slices can be transmitted and decoded in parallel without having to wait for the entire frame to be encoded.

Slices also work as resynchronization markers since the decoders can resume from the next slice (marker) when there are bit errors. On the Qualcomm Linux platform, the H.264 and HEVC encoders support slicing. A slice boundary can be specified as the number of bits per slice or the number of macroblocks per slice.

Intra refresh

The intra-refresh feature helps in reducing the channel loss in streaming and casting applications that favor a constant bit rate. The Adreno VPU supports random intra-refresh mode.

Video encoder preprocessing

In cases where a YUV frame must be rotated or flipped, applications can use the Adreno VPU to perform the rotation or flip operations before encoding the YUV frame. The rotation or flip operation is performed by the Adreno VPU without consuming extra power.

Rate control

The following table lists the supported rate control algorithms:

Rate control mode	Description
Variable bit rate (VBR)	- Minimizes the frame-by-frame video quality fluctuation. - Camcorder and Wi-Fi display are the example use cases
Constant bit rate (CBR)	- Reduces bit rate fluctuation - Used for real-time communication with channel bandwidth limitation - Videotelephony and streaming are the example use cases
Maximum bit rate (MBR)	- Limits the bit rate while maintaining flexibility and may bounce up and down within the set target - Bit rate increases when the activity in a scene increases within a maximum limit - Integrated with a smart bit allocation (SBA) feature, to achieve better quality at a lower bit rate

Long Term Reference (LTR) support

Video compression works by eliminating redundancies within the frame (intra-frame) and between the frames (inter-frame). Previously, the encoded frames that used to serve as a basis to derive future frames were known as reference frames.

Following are the two types of reference frames that allow advanced encoding applications to control how the reference frames are stored and referred:

- Short Term Reference (STR): Recent frames are maintained in a reference buffer list from the newest to the oldest. The encoder automatically manages frames using STRs for reference and deletes them from a stored list when they are no longer used.
- Long Term Reference (LTR): Frames that the application can save, use, and remove. The LTR frames are used to improve quality and ensure error resiliency in video communication. The maximum number of frames that can be marked with LTR frames depends on the device capability.

LTR frames are useful in error-prone channels. Referring to LTR in error-prone channels reduces the chance of drift error due to channel losses. The receiver must confirm that the LTR is received successfully and can request a new LTR when an error occurs. The network protocols have checksums that can confirm this, along with the decoder corruption flags. New LTR frames are generated from the sender until the receiver confirms that a successful LTR is received.

LTR frames are also useful in videos with scene changes where an LTR with the previous scene can be preserved. If that scene comes back, then the LTR can be used effectively.

At the start of the new Group of pictures (GOP), the encoder automatically fills the LTR slots, and the first slot (slot# 0) is filled with an IDR frame. However, an application can explicitly send an LTR mark request to mark the LTR frames in the appropriate slots. The following image shows the flow diagram for LTR usage, and how LTR frames can be marked and used on an H.264 or HEVC encoder:

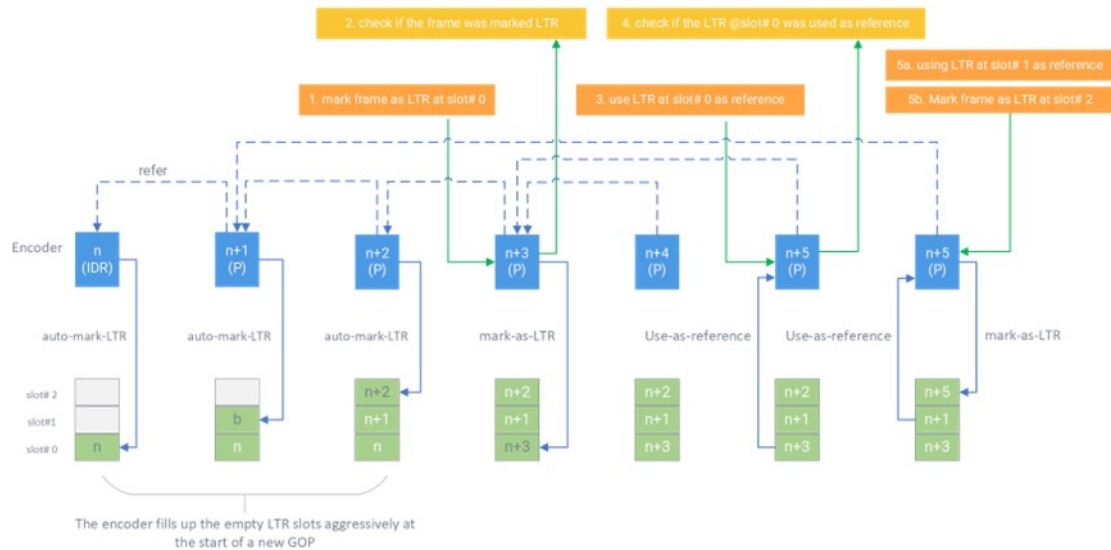


Figure 3-2 Long Term Reference encoding

Dynamic encoder properties

The Adreno VPU encoder supports dynamic change of properties like bit rate, frame rate, and sync frame. This support enables the application to seamlessly change the properties and helps in improved visual experience, video data adjusting to network conditions, and minimize the loss of data during transmission.

QCS6490 video capabilities and features

The Adreno VPU on the QCS6490 is a sixth-generation Ultra-HD video processing unit that comes with the following capabilities:

Feature	Capability
Video decode	Up to 4096 × 2160 at 60 fps for H.264/HEVC/VP9
Video encode	Up to 4096 × 2160 at 30 fps for H.264/HEVC
Video concurrency	1920 × 1088 at 60 fps decode and 1920 × 1088 at 60 fps encode 3840 × 2160 at 30 fps decode and 1920 × 1088 at 30 fps encode 4096 × 2160 at 30 fps encode and 1280 × 720 at 30 fps encode
High frame rate capture	1280 × 720 at 480 fps or 1920 × 1088 at 240 fps
Maximum macro blocks per second	2088960

Feature	Capability
	You can calculate the maximum macroblocks per second using the formula: $(\text{aligned width} \times \text{aligned height} \times \text{fps}) / (\text{macroblock size})$. For example, $(4096 \times 2176 \times 60) / 256 = 2088960$
10-bit playback	HEVC and VP9

Table 3-2 Adreno VPU decoder capabilities

Decoder standard	Supported profile and level	Minimum/Maximum resolution, Maximum frame rate and bit rate	Maximum supported resolution, frame rate, and bit rate	Limitations/tools not supported
HEVC	- Main profile 8-bit up to level 5.1 - Main profile 10-bit, up to level 5.1, HLG schemes	- Minimum resolution: 96×96 - Maximum resolution: 4096×2160 or 2160×4096 - Maximum frame rate: 480 fps - Maximum bit rate: 100 Mbps	1280×720 at 480 fps, 100 Mbps 1920×1088 at 240 fps, 100 Mbps 3840×2160 at 60 fps, 100 Mbps 4096×2160 at 60 fps, 100 Mbps	- Maximum 128 slices per frame - Individual slice-based decoding
H.264	Constrained baseline, baseline, main, high, constrained high profiles; up to level 5.2	- Minimum resolution: 96×96 - Maximum resolution: 4096×2160 or 2160×4096 - Maximum frame rate: 480 fps - Maximum bit rate: 100 Mbps	1280×720 at 480 fps, 100 Mbps 1920×1088 at 240 fps, 100 Mbps 3840×2160 at 60 fps, 100 Mbps 4096×2160 at 60 fps, 100 Mbps	- Flexible macroblock order (FMO) - Arbitrary slice ordering (ASO) - Redundant slices (RS) - Data partition - Maximum 10 slices per frame - Interlaced content is not supported
VP9	- Profile 0; 8-bit, up to level 5.1 - Profile 2; 10-bit, up to level 5.1 HLG/PQ schemes	- Minimum resolution: 96×96 - Maximum resolution: 4096×2160 or 2160×4096 - Maximum frame rate: 480 fps - Maximum bit rate: 100 Mbps	1280×720 at 480 fps, 100 Mbps 1920×1088 at 240 fps, 100 Mbps 3840×2160 at 60 fps, 100 Mbps 4096×2160 at 60 fps, 100 Mbps	Profile 2; 12-bit is not supported

Table 3-3 Adreno VPU encoder capabilities

Encoder standard	Supported profile and level and RC modes	Minimum/Maximum resolution, maximum frame rate, and maximum bit rate	Supported resolution, frame rate, bit rate	Limitations/ tools not supported
H.264	- Constrained baseline, baseline, main, high, constrained high profiles; up to level 5 - VBR, CBR, MBR	- Minimum resolution: 128 × 128 - Maximum resolution: 4096 × 2160 or 2160 × 4096 - Maximum frame rate: 240 fps - Maximum bit rate: 100 Mbps	1280 × 720 at 240 fps, 100 Mbps 1920 × 1088 at 120 fps, 100 Mbps 3840 × 2160 at 30 fps, 100 Mbps 4096 × 2160 at 30 fps, 100 Mbps	None
HEVC	- Main profile 8-bit, up to level 5.0 - Main/High tier VBR, CBR, MBR	- Minimum resolution: 128 × 128 - Maximum resolution: 4096 × 2160 or 2160 × 4096 - Maximum frame rate: 240 fps - Maximum bit rate: 100 Mbps	1280 × 720 at 240 fps, 100 Mbps 1920 × 1088 at 120 fps, 100 Mbps 3840 × 2160 at 30 fps, 100 Mbps 4096 × 2160 at 30 fps, 100 Mbps	Vertical tiling is only enabled for frame width ≥ 960

Table 3-4 Adreno VPU feature description

Feature	Description	Codecs	Remarks
Encoder input color formats	NV12 and QC08C	H.264 and HEVC	–
Decoder output color formats	NV12, QC08C, and QC10C	H.264, HEVC, and VP9	–
Rotation	Supports 90, 180, and 270-degree rotation before encoding the frame	H.264 and HEVC	Supports static rotation only
Flip	Supports horizontal and vertical flip before encoding the frame	H.264 and HEVC	Supports static and dynamic flip
B-frame encode	Up to 1920 × 1088 at 60 fps encode	H.264 and HEVC	The maximum number of B-frames supported between two P-frames is one
Hierarchical-P encode	Up to 5 layers	H.264 and HEVC	–
Initial QP override	Supported for I, P and B-frames	H.264 and HEVC	–
Slice encode	Yes	H.264 and HEVC	The slice boundary is supported based on the number of bits per slice or the number of macroblocks per slice

Table 3-4 Adreno VPU feature description (cont.)

Feature	Description	Codecs	Remarks
Intra refresh	Random refresh mode	H.264 and HEVC	- Supported only in 8-bit encoding - Supported only in the CBR RC mode
Rate control	CBR, VBR, and MBR	H.264 and HEVC	–
Long term reference (LTR)	Two frames	H.264 and HEVC	Supports only in the CBR RC mode
Dynamic properties for encoder	Sync frame, bit rate, and fps	H.264 and HEVC	–
Multichannel support	Up to 16 instances	H.264, HEVC, and VP9	Subject to maximum macroblock capability

4 Architecture

The video software stack consists of a client application that interacts with the Adreno VPU using the V4L2 framework. The following figures show the software architecture for decoder and encoder:

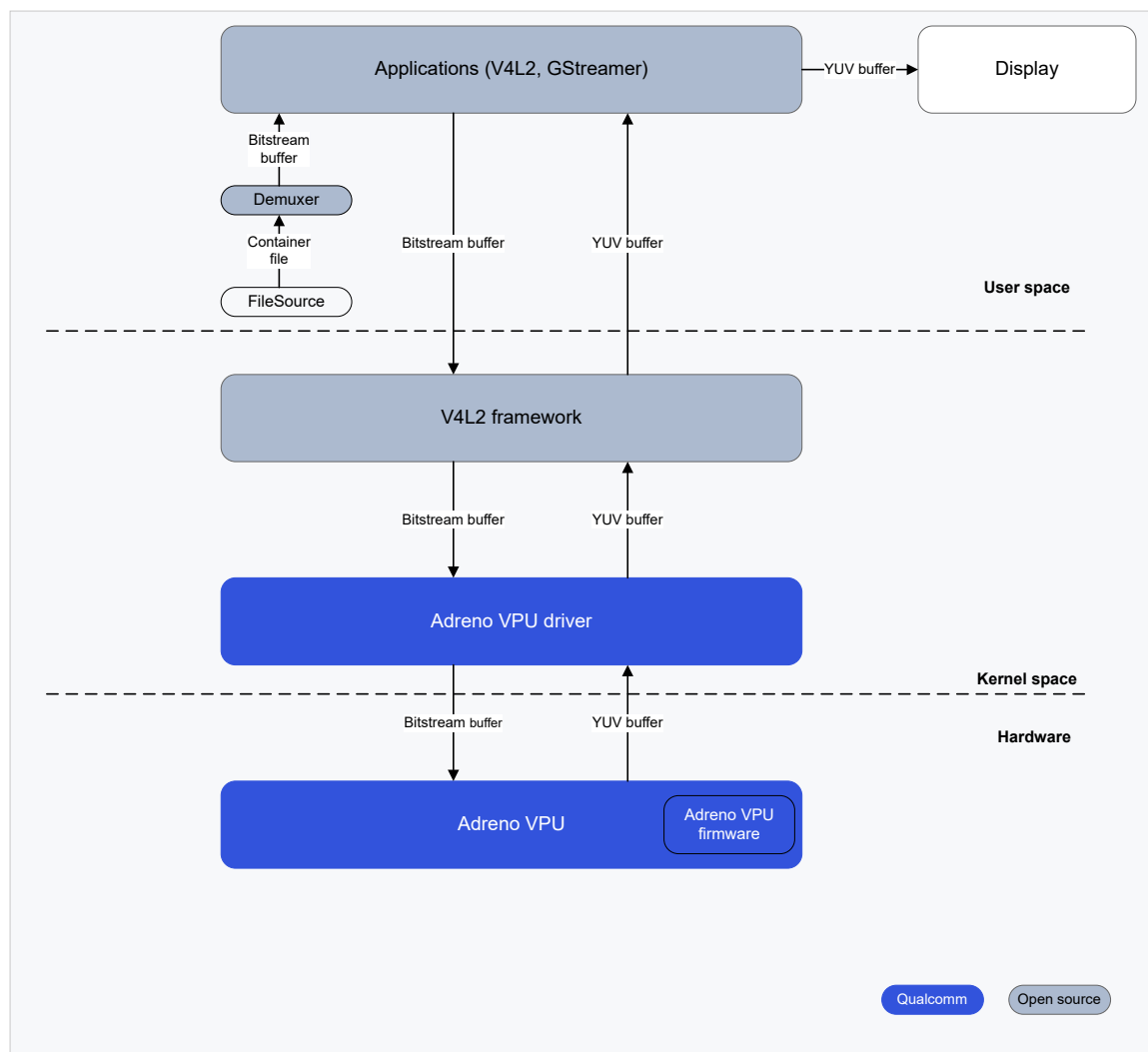


Figure 4-1 Video decoder software architecture

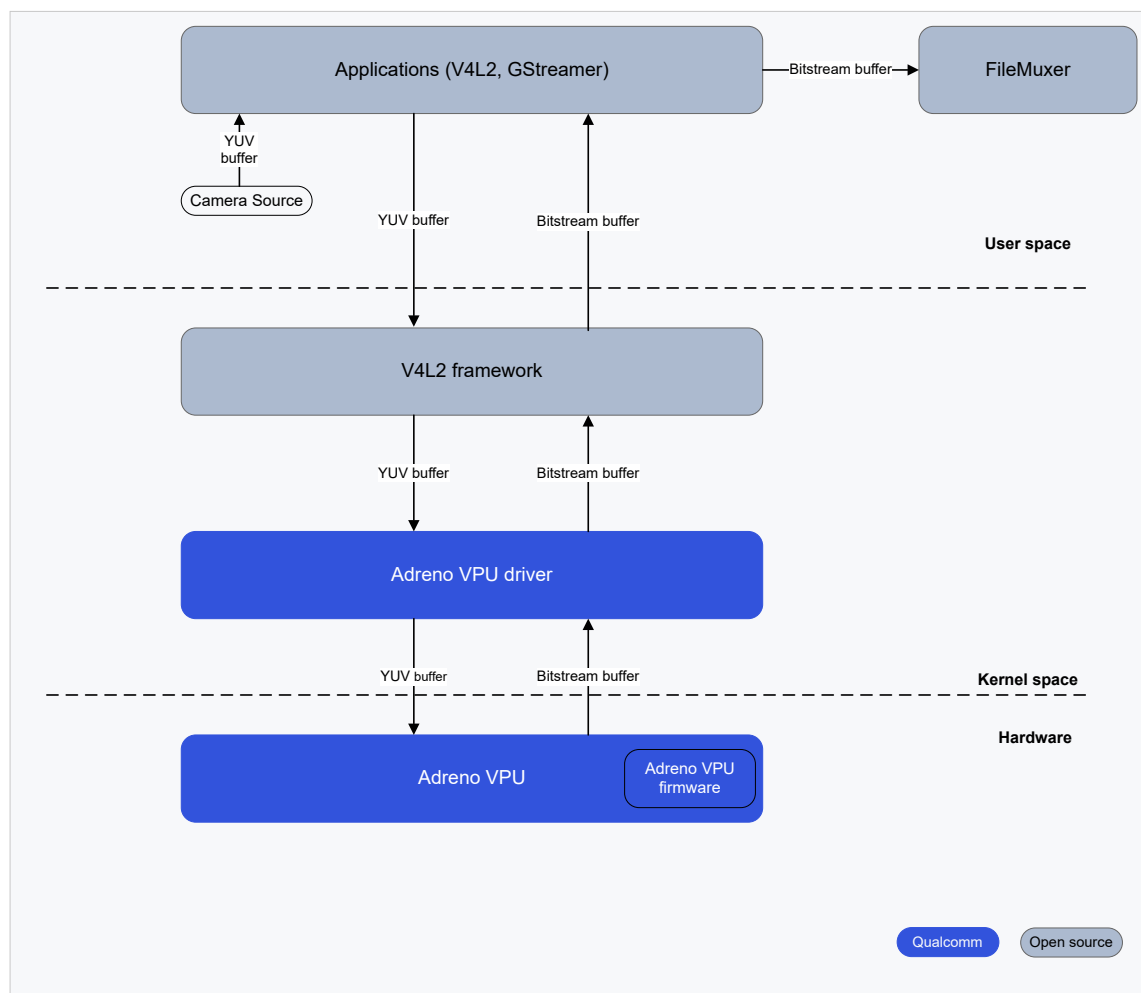


Figure 4-2 Video encoder software architecture

The following table lists the components of the video software architecture:

Component	Description
User application	Based on V4L2/GStreamer that interacts with the FileSource for decoder and the CameraSource for encoder.
V4L2 framework	- Interacts with user application and the Adreno VPU driver - Handles events, callbacks, and buffer management
Adreno VPU driver	- Acts as an entry point to the video kernel mode driver (KMD) - Common driver for the encoder and decoder that controls the Adreno VPU - Manages the video driver state machine, buffer allocation, clock/bus management, and communication with Adreno VPU using the Host Firmware Interface (HFI)
Adreno VPU	A hardware accelerated engine that performs the video decoding and encoding operations with minimal power consumption.

5 Interfaces

The Qualcomm Linux platform supports the following video APIs:

- [V4L2 APIs](#): The Adreno VPU driver follows a standard V4L2-compliant call flow. You can find information on V4L2 APIs, commands, controls, and events [here](#).
- [GStreamer APIs](#): The Qualcomm GStreamer plugins are open-source GStreamer framework compliant. You can find information about various GStreamer based plugins in the QIM SDK.

V4L2 APIs

The Adreno VPU driver is based on the V4L2 interface and supports video encode and decode for all clients that adhere to the V4L2 guidance.

This section describes V4L2 device nodes, file operations, commands, controls, and events for decoder and encoder.

VPU driver nodes

The VPU driver is registered during the bootup process and assigns the device node number. Following is the information on VPU driver nodes for decoder and encoder:

- VPU driver node for decoder: `/dev/video32`
- VPU driver node for encoder: `/dev/video33`

V4L2 file operations

The Adreno VPU driver supports the following V4L2 file operations:

Operation	Description
<code>open</code>	Opens the video device file descriptor (FD) that helps communicate with the VPU driver using the <code>ioctl</code> commands
<code>close</code>	Closes the video device at the completion of a video use case, and releases all the VPU driver resources related to the video device
<code>poll</code>	Waits for the events from the VPU driver
<code>ioctl</code>	Sends commands or controls to the VPU device

V4L2 video decoder

The Adreno VPU decoder device node, supported controls, commands, and events are described [here](#). For the steps to open a VPU decoder node, you can refer to the [VPU driver nodes](#) topic.

V4L2 ioctl commands for decoder

The descriptions for the standard `ioctl` commands are available at [Video for Linux API Function Reference](#).

A list of supported ioctl commands available in the VPU driver for the video decoder use case is as follows:

- VIDIOC_ENUM_FMT
- VIDIOC_TRY_FMT
- VIDIOC_G_FMT
- VIDIOC_S_FMT
- VIDIOC_ENUM_FRAMESIZES
- VIDIOC_REQBUFS
- VIDIOC_QUERY_BUF
- VIDIOC_CREATE_BUFS
- VIDIOC_PREPARE_BUF
- VIDIOC_QBUF
- VIDIOC_DQBUF
- VIDIOC_STREAMON
- VIDIOC_STREAMOFF
- VIDIOC_QUERYCAP
- VIDIOC_QUERYCTRL
- VIDIOC_QUERYMENU
- VIDIOC_SUBSCRIBE_EVENT
- VIDIOC_UNSUBSCRIBE_EVENT
- VIDIOC_G_SELECTION
- VIDIOC_TRY_DECODER_CMD
- VIDIOC_DECODER_CMD
- VIDIOC_G_CTRL
- VIDIOC_S_CTRL

All the mandatory commands necessary for video decoder use cases mentioned at [Video Decoder Interface](#) are supported.

V4L2 open-source controls for decoder

The descriptions for the standard open-source controls are available at [Codec Control Reference](#).

The following table lists the supported open-source controls available in the VPU driver for the video decoder use cases:

Control ID	Purpose	Dynamic/Static
V4L2_CID_MPEG_VIDEO_H264_PROFILE	H.264 decoder profile	Static
V4L2_CID_MPEG_VIDEO_HEVC_PROFILE	HEVC decoder profile	Static
V4L2_CID_MPEG_VIDEO_VP9_PROFILE	VP9 decoder profile	Static
V4L2_CID_MPEG_VIDEO_H264_LEVEL	H.264 decoder level	Static
V4L2_CID_MPEG_VIDEO_HEVC_LEVEL	HEVC decoder level	Static
V4L2_CID_MPEG_VIDEO_VP9_LEVEL	VP9 decoder level	Static
V4L2_CID_MPEG_VIDEO_HEVC_TIER	HEVC decoder tier	Static

A list of supported V4L2 open-source events for the decoder is as follows:

- V4L2_EVENT_EOS
- V4L2_EVENT_SOURCE_CHANGE
- V4L2_EVENT_CTRL

A list of supported V4L2 commands for the decoder is as follows:

- V4L2_DEC_CMD_START
- V4L2_DEC_CMD_STOP

V4L2 video encoder

The Adreno VPU encoder device node, supported controls, commands, and events are described here. For the steps to open a VPU encoder node, you can refer to the [VPU driver nodes](#) topic.

V4L2 ioctl commands for encoder

The descriptions for the standard ioctl commands are available at [Video for Linux API Function Reference](#).

A list of supported ioctl commands available in the VPU driver for the encode use cases is as follows:

- VIDIOC_ENUM_FMT
- VIDIOC_TRY_FMT
- VIDIOC_G_FMT
- VIDIOC_S_FMT
- VIDIOC_ENUM_FRAMESIZES
- VIDIOC_ENUM_FRAMEINTERVALS

- VIDIOC_REQBUFS
- VIDIOC_QUERY_BUF
- VIDIOC_CREATE_BUF
- VIDIOC_PREPARE_BUF
- VIDIOC_QBUF
- VIDIOC_DQBUF
- VIDIOC_STREAMON
- VIDIOC_STREAMOFF
- VIDIOC_QUERYCAP
- VIDIOC_QUERYCTRL
- VIDIOC_QUERYMENU
- VIDIOC_SUBSCRIBE_EVENT
- VIDIOC_UNSUBSCRIBE_EVENT
- VIDIOC_G_SELECTION
- VIDIOC_S_SELECTION
- VIDIOC_S_PARM
- VIDIOC_G_PARM
- VIDIOC_TRY_ENCODER_CMD
- VIDIOC_ENCODER_CMD
- VIDIOC_G_CTRL
- VIDIOC_S_CTRL

All the mandatory commands necessary for video encoder use cases mentioned at [Video Encoder Interface](#) are supported.

V4L2 open-source controls for encoder

The descriptions for the standard open-source controls are available at [Codec Control Reference](#).

The following table lists the supported open-source controls available in the VPU driver for the video encoder use cases:

Control ID	Purpose	Dynamic/ Static
V4L2_CID_MPEG_VIDEO_H264_PROFILE	H.264 encoder profile	Static
V4L2_CID_MPEG_VIDEO_HEVC_PROFILE	HEVC encoder profile	Static
V4L2_CID_MPEG_VIDEO_H264_LEVEL	H.264 encoder level	Static

Control ID	Purpose	Dynamic/ Static
V4L2_CID_MPEG_VIDEO_HEVC_LEVEL	HEVC encoder level	Static
V4L2_CID_MPEG_VIDEO_HEVC_TIER	H.264 encoder tier	Static
V4L2_CID_HFLIP	Horizontal flip	Both
V4L2_CID_VFLIP	Vertical flip	Both
V4L2_CID_ROTATE	Rotation	Static
V4L2_CID_MPEG_VIDEO_HEADER_MODE	Header mode	Static
V4L2_CID_MPEG_VIDEO_PREPEND_SPSPPS_TO_IDR	SPS/PPS with IDR frame	Static
V4L2_CID_MPEG_VIDEO_FORCE_KEY_FRAME	Request force sync frame	Both
V4L2_CID_MPEG_VIDEO_BITRATE	Average bit rate	Both
V4L2_CID_MPEG_VIDEO_BITRATE_PEAK	Peak bit rate	Both
V4L2_CID_MPEG_VIDEO_BITRATE_MODE	Rate control	Static
V4L2_CID_MPEG_VIDEO_FRAME_SKIP_MODE	Frame skip mode	Static
V4L2_CID_MPEG_VIDEO_FRAME_RC_ENABLE	RC enable/disable	Static
V4L2_CID_MPEG_VIDEO_GOP_SIZE	Group-of-pictures size	Both
V4L2_CID_MPEG_VIDEO_B_FRAMES	B-frames	Static
V4L2_CID_MPEG_VIDEO_LTR_COUNT	LTR frame count	Static
V4L2_CID_MPEG_VIDEO_USE_LTR_FRAMES	Use LTR	Both
V4L2_CID_MPEG_VIDEO_FRAME_LTR_INDEX	Mark LTR	Both
V4L2_CID_MPEG_VIDEO_BASELAYER_PRIORITY_ID	Set base layer ID for layer encode	Static
V4L2_CID_MPEG_VIDEO_INTRA_REFRESH_PERIOD_TYPE	Intra refresh	Static
V4L2_CID_MPEG_VIDEO_INTRA_REFRESH_PERIOD	Intra refresh period	Both

Control ID	Purpose	Dynamic/ Static
V4L2_CID_MPEG_VIDEO_H264_MIN_QP	H.264 minimum QP	Static
V4L2_CID_MPEG_VIDEO_HEVC_MIN_QP	HEVC minimum QP	Static
V4L2_CID_MPEG_VIDEO_H264_MAX_QP	H.264 maximum QP	Static
V4L2_CID_MPEG_VIDEO_HEVC_MAX_QP	HEVC maximum QP	Static
V4L2_CID_MPEG_VIDEO_H264_I_FRAME_MIN_QP	H.264 I-frame minimum QP	Static
V4L2_CID_MPEG_VIDEO_HEVC_I_FRAME_MIN_QP	HEVC I-frame minimum QP	Static
V4L2_CID_MPEG_VIDEO_H264_P_FRAME_MIN_QP	H.264 P-frame minimum QP	Static
V4L2_CID_MPEG_VIDEO_HEVC_P_FRAME_MIN_QP	HEVC P-frame minimum QP	Static
V4L2_CID_MPEG_VIDEO_H264_B_FRAME_MIN_QP	H.264 B-frame minimum QP	Static
V4L2_CID_MPEG_VIDEO_HEVC_B_FRAME_MIN_QP	HEVC B-frame minimum QP	Static
V4L2_CID_MPEG_VIDEO_H264_I_FRAME_MAX_QP	H.264 I-frame maximum QP	Static
V4L2_CID_MPEG_VIDEO_HEVC_I_FRAME_MAX_QP	HEVC I-frame maximum QP	Static
V4L2_CID_MPEG_VIDEO_H264_P_FRAME_MAX_QP	H.264 P-frame maximum QP	Static
V4L2_CID_MPEG_VIDEO_HEVC_P_FRAME_MAX_QP	HEVC P-frame maximum QP	Static
V4L2_CID_MPEG_VIDEO_H264_B_FRAME_MAX_QP	H.264 B-frame maximum QP	Static
V4L2_CID_MPEG_VIDEO_HEVC_B_FRAME_MAX_QP	HEVC B-frame maximum QP	Static
V4L2_CID_MPEG_VIDEO_HEVC_I_FRAME_QP	HEVC I-frame QP	Both
V4L2_CID_MPEG_VIDEO_H264_I_FRAME_QP	H.264 I-frame QP	Both
V4L2_CID_MPEG_VIDEO_HEVC_P_FRAME_QP	HEVC P-frame QP	Both
V4L2_CID_MPEG_VIDEO_H264_P_FRAME_QP	H.264 P-frame QP	Both
V4L2_CID_MPEG_VIDEO_HEVC_B_FRAME_QP	HEVC B-frame QP	Both

Control ID	Purpose	Dynamic/ Static
V4L2_CID_MPEG_VIDEO_H264_B_FRAME_QP	H.264 B-frame QP	Both
V4L2_CID_MPEG_VIDEO_HEVC_HIER_CODING_TYPE	HEVC Hier coding type	Static
V4L2_CID_MPEG_VIDEO_H264_HIERARCHICAL_CODING_TYPE	H.264 Hier coding type	Static
V4L2_CID_MPEG_VIDEO_H264_HIERARCHICAL_CODING	Enables H.264 Hier coding	Static
V4L2_CID_MPEG_VIDEO_HEVC_HIER_CODING_LAYER	HEVC Hier layer count	Both
V4L2_CID_MPEG_VIDEO_H264_HIERARCHICAL_CODING_LAYER	H.264 Hier layer count	Both
V4L2_CID_MPEG_VIDEO_H264_HIER_CODING_L0_BR	H.264 layer 0-bit rate	Both
V4L2_CID_MPEG_VIDEO_HEVC_HIER_CODING_L0_BR	HEVC layer 0-bit rate	Both
V4L2_CID_MPEG_VIDEO_H264_HIER_CODING_L1_BR	H.264 layer 1-bit rate	Both
V4L2_CID_MPEG_VIDEO_HEVC_HIER_CODING_L1_BR	HEVC layer 1-bit rate	Both
V4L2_CID_MPEG_VIDEO_H264_HIER_CODING_L2_BR	H.264 layer 2-bit rate	Both
V4L2_CID_MPEG_VIDEO_HEVC_HIER_CODING_L2_BR	HEVC layer 2-bit rate	Both
V4L2_CID_MPEG_VIDEO_H264_HIER_CODING_L3_BR	H.264 layer 3-bit rate	Both
V4L2_CID_MPEG_VIDEO_HEVC_HIER_CODING_L3_BR	HEVC layer 3-bit rate	Both
V4L2_CID_MPEG_VIDEO_H264_HIER_CODING_L4_BR	H.264 layer 4-bit rate	Both
V4L2_CID_MPEG_VIDEO_HEVC_HIER_CODING_L4_BR	HEVC layer 4-bit rate	Both
V4L2_CID_MPEG_VIDEO_H264_HIER_CODING_L5_BR	H.264 layer 5-bit rate	Both
V4L2_CID_MPEG_VIDEO_HEVC_HIER_CODING_L5_BR	HEVC layer 5-bit rate	Both
V4L2_CID_MPEG_VIDEO_H264_ENTROPY_MODE	H.264 entropy mode	Static
V4L2_CID_MPEG_VIDEO_MULTI_SLICE_MODE	Encode slice mode	Static

Control ID	Purpose	Dynamic/ Static
V4L2_CID_MPEG_VIDEO_MULTI_SLICE_MAX_BYTES	Maximum bytes in slice	Static
V4L2_CID_MPEG_VIDEO_MULTI_SLICE_MAX_MB	Maximum macroblocks in slice	Static

All static configuration options must be set after setting the formats on both the planes and before requesting buffers on either of the planes. The dynamic parameters can be called after setting the formats on both the planes.

For more information on the V4L2 controls for encoder, see [Codec Control Reference](#).

A list of supported open-source events for the encoder is as follows:

- V4L2_EVENT_EOS
- V4L2_EVENT_CTRL

A list of supported open-source commands for the encoder is as follows:

- V4L2_ENC_CMD_START
- V4L2_ENC_CMD_STOP

GStreamer APIs

Qualcomm provides you with the GStreamer-based plugins. You can add these plugins to your media pipeline. For more information on the Qualcomm provided GStreamer-based plugins, see [Qualcomm GST plugins](#).

For more information on the GStreamer application development and pipeline creation, see [Building an Application](#).

6 Software

The video software modules are compiled as part of the entire software build package, and the corresponding binaries are available in the `meta-qcom-multimedia` image. For more information on the steps to sync and build the `meta-qcom-multimedia` image, see [Qualcomm Linux Build Guide](#). Information related to the video software, V4L2 build instructions, and the GStreamer plugins is provided here.

V4L2 for video

The following is the information related to the source tree and build instructions for the VPU driver, and the commands that help push the video firmware.

Source tree

The video software modules include the Adreno VPU driver, the video device tree, and the firmware binary.

- Adreno VPU driver BitBake recipe file

Product	VPU driver BitBake recipe file path
QCS6490	<workspace-path>/layers/meta-qcom-hwe/recipes-multimedia/video/videodlkm_1.0.bb

- Video device tree BitBake recipe file: The video device tree node is a part of the root device tree file.

Product	Device tree BitBake recipe file path
QCS6490	<workspace-path>/layers/meta-qcom-hwe/recipes-multimedia/video / videodtb_1.0.bb

VPU driver build instructions

Use `devtool` to add the debug changes and compile the VPU driver (`iris_vpu.ko`). Following are the instructions:

- The source code for VPU drivers is available for access with the following BitBake command:

```
devtool modify videodlkm
```

You can make the required debug changes to the VPU driver code that is synced at: `<workspace-path>/build-qcom-wayland/workspace/sources/videodlkm`. After making the debug changes, use the following command to build the VPU driver:

```
devtool build videodlkm
```

- After the compilation is complete, the corresponding `iris_vpu.ko` is created at: `<workspace-path>/build-qcom-wayland/workspace/sources/videodlkm/iris_vpu.ko`.

- Push the compiled library to the device using the following command after you enter into the SSH shell:

NOTE Before an SSH command is run, ensure that you enable SSH in the Permissive mode. For instructions, see [How to SSH](#).

```
mount -o rw,remount /
```

Change the following command according to your <workspace-path> and the <IP address of the device> and run on the host:

```
scp <workspace-path>/build-qcom-wayland/workspace/sources/videodlkm/iris_vpu.ko root@<IP address of the device>/lib/modules/6.6.28/updates/
```

NOTE When prompted for a password, enter oelinux123.

- After executing the preceding commands, power Off and power On the target to reflect the updated debug changes.

Video device tree

The video device tree node is a part of the root device tree file. The source code for VPU device tree is available for access with the following BitBake command:

```
devtool modify videodtb
```

The VPU device tree code is synced at: <workspace-path>/build-qcom-wayland/workspace/sources/videodtb

Video firmware

The video firmware binary is a prebuilt image from Qualcomm. Use the following command to push the video firmware binary to your device:

Product	Command
QCS6490	<pre>cd <workspace-path>/build-qcom-wayland/tmp-glibc/work/qcm6490-qcom-linux/qcom-multimedia-image/1.0-r0/rootfs/lib/firmware/qcom/vpu-2.0/ scp vpu20_1v.mbn root@<IP address of the device>/lib/firmware/qcom/vpu-2.0/ scp vpu20_1v_unsigned.mbn root@<IP address of the device>/lib/firmware/qcom/vpu-2.0/</pre>

NOTE When prompted for a password, enter oelinux123.

GStreamer plugins for video

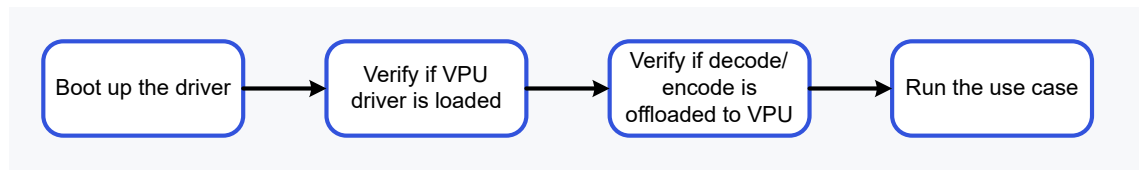
The GStreamer plugins for video decoder and encoder are part of the QIM SDK. Download the entire QIM SDK to use the v4l2h264dec, v4l2h265dec, v4l2h264enc, v4l2h265enc, and the v4l2vp9dec plugins. For more information on the steps to download and build the QIM SDK, see [Qualcomm Linux Intelligent Multimedia SDK \(QIM SDK\) Quick Start Guide](#).

The following table lists the GStreamer plugins and their corresponding reference links:

Plug-in	Description/Reference link
v4l2h264dec	Decode the H.264 video stream
v4l2h265dec	Decode the H.265 video stream
v4l2h264enc	Encode the H.264 video stream

Plug-in	Description/Reference link
v4l2h265enc	Encode the H.265 video stream
v4l2vp9dec	Decode the VP9 video stream

7 Verify



The commands and samples provided here enable you to verify the following:

- VPU driver is successfully loaded
- Decoding/encoding is offloaded to the VPU

Verify if the VPU driver is successfully loaded

The VPU driver is loaded as part of the kernel image. You can confirm whether the VPU driver is loaded successfully by executing the following methods:

Method1:

- Execute the following `lsmod` command on the device shell:

```
lsmod | grep -i "iris"
```

- The following is the sample output that helps you verify if the VPU driver is successfully loaded. Ensure that the output shows an entry for `iris_vpu` under the Module column:

Module	Size	Used by
iris_vpu	585728	0

Method2:

- Execute the `udevadm info -n /dev/video*` command on the device shell and verify if the output shows `msm_vidc_decoder` and `msm_vidc_encoder` against the `ID_V4L_PRODUCT` tag.

```
udevadm info -n /dev/video*
```

- The following sample output helps you verify whether the VPU is successfully loaded:

```
P: /devices/platform/soc@0/aa00000.video-codec/video4linux/video32
N: video32
L: 0
S: v4l/by-path/platform-aa00000.video-codec-video-index0
E: DEVPATH=/devices/platform/soc@0/aa00000.video-codec/video4linux/video32
```

```

E: DEVNAME=/dev/video32
E: MAJOR=81
E: MINOR=21
E: SUBSYSTEM=video4linux
E: USEC_INITIALIZED=6369466
E: ID_V4L_VERSION=2
E: ID_V4L_PRODUCT=msm_vidc_decoder
E: ID_V4L_CAPABILITIES=:
E: ID_PATH=platform-aa00000.video-codec
E: ID_PATH_TAG=platform-aa00000_video-codec
E: ID_FOR_SEAT=video4linux-platform-aa00000_video-codec
E: DEVLINKS=/dev/v4l/by-path/platform-aa00000.video-codec-video-index0
E: TAGS=:uaccess:seat:
E: CURRENT_TAGS=:uaccess:seat:

P: /devices/platform/soc@0/aa00000.video-codec/video4linux/video33
N: video33
L: 0
S: v4l/by-path/platform-aa00000.video-codec-video-index1
E: DEVPATH=/devices/platform/soc@0/aa00000.video-codec/video4linux/video33
E: DEVNAME=/dev/video33
E: MAJOR=81
E: MINOR=22
E: SUBSYSTEM=video4linux
E: USEC_INITIALIZED=6363137
E: ID_V4L_VERSION=2
E: ID_V4L_PRODUCT=msm_vidc_encoder
E: ID_V4L_CAPABILITIES=:
E: ID_PATH=platform-aa00000.video-codec
E: ID_PATH_TAG=platform-aa00000_video-codec
E: ID_FOR_SEAT=video4linux-platform-aa00000_video-codec
E: DEVLINKS=/dev/v4l/by-path/platform-aa00000.video-codec-video-index1
E: TAGS=:uaccess:seat:
E: CURRENT_TAGS=:uaccess:seat:

```

NOTE To know the device nodes, check for the `DEVNAME` tag. In the earlier example, the `/dev/video32` is for the VPU decoder node and `/dev/video33` is for the VPU encoder node.

Verify if the decoding/encoding is offloaded to the VPU

You can confirm if the video decoding or encoding is successfully offloaded to the VPU by doing the following:

- Execute the following command as a loop:


```
cat /proc/interrupts | grep "msm-vidc"
```

- The following is the sample output that helps you verify whether the video decoding/encoding is offloaded to the VPU. Verify if the interrupt count from `msm-vidc` is increasing across any CPU when the use case is running:

```
253: 4      0      0      0      0      0      0      0      0      GICv3
206 Level  msm-vidc
```

8 Configure

Information on how to configure the V4L2 and GStreamer APIs for different features is provided here.

Configure V4L2 APIs for different features

The following information is about V4L2 control IDs, accepted format, ranges, and sample code for different features:

B-frame encode

You can configure the VPU driver to enable B-frames in an encoder session using the `VIDIOC_S_CTRL` ioctl system call. This configuration must be set before the `VIDIOC_STREAMON` on an output plane. The VPU driver supports B-frame encode only with the Hierarchical coding. The Dynamic setting is not supported.

Control ID for H.264 and HEVC codec	Accepted format	Range
<code>V4L2_CID_MPEG_VIDEO_B_FRAMES</code>	Integer	▪ {0, 1} ▪ 0: Disable B-frame ▪ 1: Enable one B-frame

The following code sample shows the static configuration to enable a single B-frame:

```
v4l2_control control;
int ret = 0;
int bframes = 1;
memset(&control, 0, sizeof(control));
control.id = V4L2_CID_MPEG_VIDEO_B_FRAMES;
control.value = bframes;
ret = ioctl(mDriverFd, VIDIOC_S_CTRL, &control); // mDriverFd returned from
open() system call
if (ret) {
    return -1;
}
memset(&control, 0, sizeof(control));
control.id = V4L2_CID_MPEG_VIDEO_H264_HIERARCHICAL_CODING_TYPE;
control.value = 0; // V4L2_MPEG_VIDEO_HEVC_HIERARCHICAL_CODING_B
ret = ioctl(mDriverFd, VIDIOC_S_CTRL, &control); // mDriverFd returned from
open() system call
if (ret) {
    return -1;
}
```

```
memset(&control, 0, sizeof(control));
control.id = V4L2_CID_MPEG_VIDEO_H264_HIERARCHICAL_CODING_LAYER;
control.value = 1; //Enable 1 layer
ret = ioctl(mDriverFd, VIDIOC_S_CTRL, &control); // mDriverFd returned from
open() system call
if (ret) {
    return -1;
}
```

Encoder initial QP override

You can configure the VPU driver to override the initial frame Quantization Parameter (QP) for I, P, and B frames to an intended value of encoded frames. This is performed using the `VIDIOC_S_CTRL` `ioctl` system call. It also has the flexibility to set initial frame QP for I only, or P only, or B only, or I and P only, or P and B only and so on. This configuration must be set before the `VIDIOC_STREAMON` on an output plane and is only applicable to an encoder session.

Control ID for H.264 codec	Accepted format	QP range
V4L2_CID_MPEG_VIDEO_H264_I_FRAME_QP	Integer	Example according to the codec standard: 1 to 51 for 8-bit 1 to 63 (-12 to 51) for 10-bit
V4L2_CID_MPEG_VIDEO_H264_P_FRAME_QP	Integer	
V4L2_CID_MPEG_VIDEO_H264_B_FRAME_QP	Integer	

Control ID for HEVC codec	Accepted format	QP range
V4L2_CID_MPEG_VIDEO_HEVC_I_FRAME_QP	Integer	Example according to the codec standard: 1 to 51 for 8-bit 1 to 63 (-12 to 51) for 10-bit
V4L2_CID_MPEG_VIDEO_HEVC_P_FRAME_QP	Integer	
V4L2_CID_MPEG_VIDEO_HEVC_B_FRAME_QP	Integer	

The following code sample shows the static configuration parameter to choose the following:

- 40 as an initial QP for I frame
- 35 as an initial QP for P-frame
- B-frame for H.264 session

```
v4l2_control control;
int ret = 0;
memset(&control, 0, sizeof(control));
control.id = V4L2_CID_MPEG_VIDEO_H264_I_FRAME_QP;
control.value = 40;
ret = ioctl(mDriverFd, VIDIOC_S_CTRL, &control); // mDriverFd returned from
open() system call
if (ret) {
    return -1;
}
```

```

}
memset(&control, 0, sizeof(control));
control.id = V4L2_CID_MPEG_VIDEO_H264_P_FRAME_QP;
control.value = 35;
ret = ioctl(mDriverFd, VIDIOC_S_CTRL, &control); // mDriverFd returned from
open() system call
if (ret) {
    return -1;
}
memset(&control, 0, sizeof(control));
control.id = V4L2_CID_MPEG_VIDEO_H264_B_FRAME_QP;
control.value = 35;
ret = ioctl(mDriverFd, VIDIOC_S_CTRL, &control); // mDriverFd returned from
open() system call
if (ret) {
    return -1;
}
}

```

Slice encode

You can configure the VPU driver to encode a single frame to multiple slices using the `VIDIOC_S_CTRL` `ioctl` system call. A slice boundary is specified as the number of bits per slice or the number of macroblocks per slice. This configuration must be set before the `VIDIOC_STREAMON` on an output plane. Dynamic configuration is not supported in this feature, and this is applicable only to an encoder session.

Control ID for H.264 and HEVC	Accepted format	Range
V4L2_CID_MPEG_VIDEO_MULTI_SLICE_MODE	enum v4l2_mpeg_video_multi_slice_mode	Values according to the codec standard: enum v4l2_mpeg_video_multi_slice_mode
V4L2_CID_MPEG_VIDEO_MULTI_SLICE_MAX_BYTES	Integer	512 - MAX_BITRATE/8 Unit: bits/second
V4L2_CID_MPEG_VIDEO_MULTI_SLICE_MAX_MB	Integer	1 to 36864 Unit: Macroblocks

The following code sample shows the static configuration to encode frames with multiple slices and with the maximum number of macroblocks per slice set to 400:

```

v4l2_control control;
int ret = 0;
v4l2_mpeg_video_multi_slice_mode slicemode =
V4L2_MPEG_VIDEO_MULTI_SLICE_MODE_MAX_MB;
memset(&control, 0, sizeof(control));
control.id = V4L2_CID_MPEG_VIDEO_MULTI_SLICE_MODE;
control.value = slicemode;
ret = ioctl(mDriverFd, VIDIOC_S_CTRL, &control); // mDriverFd returned from
open() system call

```

```

if (ret) {
    return -1;
}
memset(&control, 0, sizeof(control));
control.id = V4L2_CID_MPEG_VIDEO_MULTI_SLICE_MAX_MB;
control.value = 400;
ret = ioctl(mDriverFd, VIDIOC_S_CTRL, &control); // mDriverFd returned from
open() system call
if (ret) {
    return -1;
}

```

Intra refresh

You can configure the VPU driver to enable the Intra refresh mode and intra refresh period using the VIDIOC_S_CTRL ioctl system call. This configuration must be set before the VIDIOC_STREAMON on an output plane. It is to be noted that the intra refresh period can be modified dynamically during the session and this feature is applicable only to an encoder session.

Control ID for H.264 and HEVC	Accepted format	Range
V4L2_CID_MPEG_VIDEO_INTRA_REFRESH_PERIOD_TYPE	enum v4l2_mpeg_video_intra_refresh_period_type	{V4L2_CID_MPEG_VIDEO_INTRA_REFRESH_PERIOD_TYPE_RANDOM, V4L2_CID_MPEG_VIDEO_INTRA_REFRESH_PERIOD_TYPE_CYCLIC} Random mode: V4L2_CID_MPEG_VIDEO_INTRA_REFRESH_PERIOD_TYPE_RANDOM Cyclic mode: V4L2_CID_MPEG_VIDEO_INTRA_REFRESH_PERIOD_TYPE_CYCLIC
V4L2_CID_MPEG_VIDEO_INTRA_REFRESH_PERIOD	Integer	Set the intra refresh period to be less than an I-frame interval Unit: number of frames

The following code sample shows the static configuration to enable the random intra refresh for every 10 frames:

```

v4l2_control control;
int ret = 0;
v4l2_mpeg_video_intra_refresh_period_type refreshtype =
V4L2_CID_MPEG_VIDEO_INTRA_REFRESH_PERIOD_TYPE_RANDOM;

memset(&control, 0, sizeof(control));
control.id = V4L2_CID_MPEG_VIDEO_INTRA_REFRESH_PERIOD_TYPE;
control.value = refreshtype;
ret = ioctl(mDriverFd, VIDIOC_S_CTRL, &control); // mDriverFd returned from

```

```

open() system call
if (ret) {
    return -1;
}
memset(&control, 0, sizeof(control));
control.id = V4L2_CID_MPEG_VIDEO_INTRA_REFRESH_PERIOD;
control.value = 10;
ret = ioctl(mDriverFd, VIDIOC_S_CTRL, &control); // mDriverFd returned from
open() system call
if (ret) {
    return -1;
}

```

Rotation

You can configure the VPU driver to rotate the input frame before encoding using the `VIDIOC_S_CTRL` ioctl system call. This configuration must be set before the `VIDIOC_STREAMON` on an output plane. This feature does not support dynamic configuration and is applicable only to an encoder session.

Control ID for H264 and HEVC	Accepted format	Range
V4L2_CID_ROTATE	Integer	{90, 180, 270} Unit: degrees

The following sample code shows the static configuration for rotating an input frame to 90 degrees:

```

v4l2_control control;
int ret = 0;
memset(&control, 0, sizeof(control));
control.id = V4L2_CID_ROTATE;
control.value = 90;
ret = ioctl(mDriverFd, VIDIOC_S_CTRL, &control); // mDriverFd returned from
open() system call
if (ret) {
    return -1;
}

```

Frame QP range

You can configure the VPU driver to set the minimum and maximum frame QP range parameter for I, P, and B frames to the intended values for encoded frames using the `VIDIOC_S_CTRL` ioctl system call. This configuration must be set before the `VIDIOC_STREAMON` on an output plane. The values are valid for an entire session. Dynamic configuration is not supported, and this feature is applicable only to an encoder session.

Control ID for H.264 codec	Accepted format	Range
V4L2_CID_MPEG_VIDEO_H264_I_FRAME_MIN_QP	Integer	Example according to the codec standard: 1 to 51 for 8-bit
V4L2_CID_MPEG_VIDEO_H264_I_FRAME_MAX_QP		

Control ID for H.264 codec	Accepted format	Range
V4L2_CID_MPEG_VIDEO_H264_P_FRAME_MIN_QP		
V4L2_CID_MPEG_VIDEO_H264_P_FRAME_MAX_QP		
V4L2_CID_MPEG_VIDEO_H264_B_FRAME_MIN_QP		
V4L2_CID_MPEG_VIDEO_H264_B_FRAME_MAX_QP		
V4L2_CID_MPEG_VIDEO_HEVC_I_FRAME_MIN_QP		
V4L2_CID_MPEG_VIDEO_HEVC_I_FRAME_MAX_QP		
V4L2_CID_MPEG_VIDEO_HEVC_P_FRAME_MIN_QP		
V4L2_CID_MPEG_VIDEO_HEVC_P_FRAME_MAX_QP		
V4L2_CID_MPEG_VIDEO_HEVC_B_FRAME_MIN_QP		
V4L2_CID_MPEG_VIDEO_HEVC_B_FRAME_MAX_QP		

The following code sample shows the static configuration parameter to choose 40 as the maximum QP and 10 as the minimum QP for the I, P, and B-frames in a H.264 session:

```
v4l2_control control;
int ret = 0;
memset(&control, 0, sizeof(control));
control.id = V4L2_CID_MPEG_VIDEO_H264_I_FRAME_MIN_QP;
control.value = 10;
ret = ioctl(mDriverFd, VIDIOC_S_CTRL, &control); // mDriverFd returned from
open() system call
if (ret) {
    return -1;
}
memset(&control, 0, sizeof(control));
control.id = V4L2_CID_MPEG_VIDEO_H264_P_FRAME_MIN_QP;
control.value = 10;
ret = ioctl(mDriverFd, VIDIOC_S_CTRL, &control); // mDriverFd returned from
open() system call
if (ret) {
    return -1;
}
memset(&control, 0, sizeof(control));
control.id = V4L2_CID_MPEG_VIDEO_H264_B_FRAME_MIN_QP;
control.value = 10;
```

```

ret = ioctl(mDriverFd, VIDIOC_S_CTRL, &control); // mDriverFd returned from
open() system call
if (ret) {
    return -1;
}
memset(&control, 0, sizeof(control));
control.id = V4L2_CID_MPEG_VIDEO_H264_I_FRAME_MAX_QP;
control.value = 40;
ret = ioctl(mDriverFd, VIDIOC_S_CTRL, &control); // mDriverFd returned from
open() system call
if (ret) {
    return -1;
}
memset(&control, 0, sizeof(control));
control.id = V4L2_CID_MPEG_VIDEO_H264_P_FRAME_MAX_QP;
control.value = 40;
ret = ioctl(mDriverFd, VIDIOC_S_CTRL, &control); // mDriverFd returned from
open() system call
if (ret) {
    return -1;
}
memset(&control, 0, sizeof(control));
control.id = V4L2_CID_MPEG_VIDEO_H264_B_FRAME_MAX_QP;
control.value = 40;
ret = ioctl(mDriverFd, VIDIOC_S_CTRL, &control); // mDriverFd returned from
open() system call
if (ret) {
    return -1;
}

```

Alternatively, you can configure the following controls if you wish to have the same minimum and maximum QP value for I, P, and B-frames:

Control ID for H.264 codec	Accepted format	Range
V4L2_CID_MPEG_VIDEO_H264_MIN_QP	Integer	Example according to the code standard: 1 to 51 for 8-bit
V4L2_CID_MPEG_VIDEO_H264_MAX_QP		

Control ID for HEVC codec	Accepted format	Range
V4L2_CID_MPEG_VIDEO_HEVC_MIN_QP	Integer	Example according to the code standard: 1 to 51 for 8-bit
V4L2_CID_MPEG_VIDEO_HEVC_MAX_QP	Integer	

Configure GStreamer APIs for different features

For more information on the GStreamer element property for different features and respective plugins, see [Qualcomm GST plugins](#).

9 Debug

The debug methods available for the VPU and GStreamer plugins are mentioned here.

Debug VPU

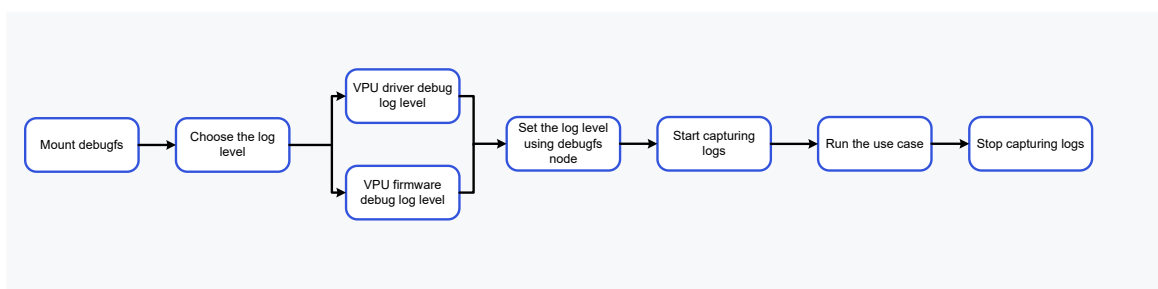


Figure 9-1 Debug VPU workflow

Following are the steps to debug the VPU driver and VPU firmware:

1. Mount the `debugfs` node, using the following command:

```
mount -t debugfs none /sys/kernel/debug/
```
2. Choose VPU driver debug log level and debug log value as listed in the following table. You can also select multiple debug log levels by using the bitmask of debug log values.

Debug log level	Debug log value
VIDC_ERR	0x00000001
VIDC_HIGH	0x00000002
VIDC_LOW	0x00000004
VIDC_PERF	0x00000008
VIDC_PKT	0x00000010
VIDC_BUS	0x00000020
VIDC_STAT	0x00000040
VIDC_ENCODER	0x00000100
VIDC_DECODER	0x00000200

Debug log level	Debug log value
VIDC_PRINTK	0x10000000
VIDC_FTRACE	0x20000000

NOTE The VPU driver debug logs are limited to error scenarios, and the default debug level is set to VIDC_ERR.

3. Set the VPU driver debug log value to the `/sys/module/iris_vpu/parameters/msm_vidc_debug` node.
 - a. The following is a sample command to enable all message levels:


```
echo 0x1000037F > /sys/module/iris_vpu/parameters/msm_vidc_debug
```
4. Choose VPU firmware debug log level and debug log value as listed in the following table. You can also select multiple debug log levels by using the bitmask of debug log values.

Debug log level	Debug log value
FW_LOW	0x00000001
FW_MED	0x00000002
FW_HIGH	0x00000004
FW_ERROR	0x00000008
FW_FATAL	0x00000010
FW_PERF	0x00000020
FW_CACHE_LOW	0x00000100
FW_CACHE_MED	0x00000200
FW_CACHE_HIGH	0x00000400
FW_CACHE_ERROR	0x00000800
FW_CACHE_FATAL	0x00001000
FW_CACHE_PERF	0x00002000
FW_PRINTK	0x10000000
FW_FTRACE	0x20000000

NOTE The VPU firmware debug logs are limited to error and fatal scenarios, and the default debug level is set to FW_ERROR | FW_FATAL.

5. Set the VPU firmware debug log value to the `/sys/module/iris_vpu/parameters/msm_fw_debug` node.
 - a. The following is a sample command to enable all message levels:


```
echo 0x1000037F > /sys/module/iris_vpu/parameters/msm_fw_debug
```
6. Start capturing kernel messages for both the VPU driver and the firmware using the following command and run the usecase:


```
cat /proc/kmsg
```

Debug GStreamer plugins

Qualcomm GStreamer video plugins are compliant with the standard GStreamer framework. For information on the debug methods for the GStreamer plugins, see [Debug GStreamer Plugins](#).

10 References

10.1 Related documents

Title	Number
Qualcomm Technologies, Inc.	
<i>Qualcomm Linux Intelligent Multimedia Software Development Kit (QIM SDK) Reference</i>	80-70014-50
<i>Qualcomm Linux Intelligent Multimedia SDK (QIM SDK) Quick Start Guide</i>	80-70014-51
Resources	
https://github.com/quic/v4l-video-test-app	
https://www.kernel.org/doc/html/v4.9/media/uapi/v4l/v4l2.html	
https://www.kernel.org/doc/html/v4.9/media/uapi/v4l/user-func.html	
https://www.kernel.org/doc/html/v5.5/media/uapi/v4l/ext-ctrls-codec.html	
https://www.kernel.org/doc/html/latest/userspace-api/media/v4l/dev-decoder.html	
https://www.kernel.org/doc/html/latest/userspace-api/media/v4l/dev-encoder.html	

10.2 Acronyms and terms

Acronym or term	Definition
ASO	Arbitrary slice ordering
CBR	Constant bit rate
FD	File descriptor
FMO	Flexible macroblock order
GOB	Group of blocks
GOP	Group of pictures
GPU	Graphics processing unit
HEVC	High efficiency video coding
HFI	Host firmware interface
LTR	Long term reference
MBR	Maximum bit rate
QP	Quantization parameter

Acronym or term	Definition
RS	Redundant slices
STR	Short term reference
VBR	Variable bit rate
VPU	Video processing unit
VT	Videotelephony
YUV	Luminance, bandwidth, chrominance, also known as YCbCr and YPbPr

LEGAL INFORMATION

Your access to and use of this material, along with any documents, software, specifications, reference board files, drawings, diagnostics and other information contained herein (collectively this “Material”), is subject to your (including the corporation or other legal entity you represent, collectively “You” or “Your”) acceptance of the terms and conditions (“Terms of Use”) set forth below. If You do not agree to these Terms of Use, you may not use this Material and shall immediately destroy any copy thereof.

1) Legal Notice.

This Material is being made available to You solely for Your internal use with those products and service offerings of Qualcomm Technologies, Inc. (“Qualcomm Technologies”), its affiliates and/or licensors described in this Material, and shall not be used for any other purposes. If this Material is marked as “Qualcomm Internal Use Only”, no license is granted to You herein, and You must immediately (a) destroy or return this Material to Qualcomm Technologies, and (b) report Your receipt of this Material to qualcomm.support@qti.qualcomm.com. This Material may not be altered, edited, or modified in any way without Qualcomm Technologies’ prior written approval, nor may it be used for any machine learning or artificial intelligence development purpose which results, whether directly or indirectly, in the creation or development of an automated device, program, tool, algorithm, process, methodology, product and/or other output. Unauthorized use or disclosure of this Material or the information contained herein is strictly prohibited, and You agree to indemnify Qualcomm Technologies, its affiliates and licensors for any damages or losses suffered by Qualcomm Technologies, its affiliates and/or licensors for any such unauthorized uses or disclosures of this Material, in whole or part.

Qualcomm Technologies, its affiliates and/or licensors retain all rights and ownership in and to this Material. No license to any trademark, patent, copyright, mask work protection right or any other intellectual property right is either granted or implied by this Material or any information disclosed herein, including, but not limited to, any license to make, use, import or sell any product, service or technology offering embodying any of the information in this Material.

THIS MATERIAL IS BEING PROVIDED “AS IS” WITHOUT WARRANTY OF ANY KIND, WHETHER EXPRESSED, IMPLIED, STATUTORY OR OTHERWISE. TO THE MAXIMUM EXTENT PERMITTED BY LAW, QUALCOMM TECHNOLOGIES, ITS AFFILIATES AND/OR LICENSORS SPECIFICALLY DISCLAIM ALL WARRANTIES OF TITLE, MERCHANTABILITY, NON-INFRINGEMENT, FITNESS FOR A PARTICULAR PURPOSE, SATISFACTORY QUALITY, COMPLETENESS OR ACCURACY, AND ALL WARRANTIES ARISING OUT OF TRADE USAGE OR OUT OF A COURSE OF DEALING OR COURSE OF PERFORMANCE. MOREOVER, NEITHER QUALCOMM TECHNOLOGIES, NOR ANY OF ITS AFFILIATES AND/OR LICENSORS, SHALL BE LIABLE TO YOU OR ANY THIRD PARTY FOR ANY EXPENSES, LOSSES, USE, OR ACTIONS HOWSOEVER INCURRED OR UNDERTAKEN BY YOU IN RELIANCE ON THIS MATERIAL.

Certain product kits, tools and other items referenced in this Material may require You to accept additional terms and conditions before accessing or using those items.

Technical data specified in this Material may be subject to U.S. and other applicable export control laws. Transmission contrary to U.S. and any other applicable law is strictly prohibited.

Nothing in this Material is an offer to sell any of the components or devices referenced herein.

This Material is subject to change without further notification.

In the event of a conflict between these Terms of Use and the *Website Terms of Use* on www.qualcomm.com or the *Qualcomm Privacy Policy* referenced on www.qualcomm.com, these Terms of Use will control. In the event of a conflict between these Terms of Use and any other agreement (written or click-through, including, without limitation any non-disclosure agreement) executed by You and Qualcomm Technologies or a Qualcomm Technologies affiliate and/or licensor with respect to Your access to and use of this Material, the other agreement will control.

These Terms of Use shall be governed by and construed and enforced in accordance with the laws of the State of California, excluding the U.N. Convention on International Sale of Goods, without regard to conflict of laws principles. Any dispute, claim or controversy arising out of or relating to these Terms of Use, or the breach or validity hereof, shall be adjudicated only by a court of competent jurisdiction in the county of San Diego, State of California, and You hereby consent to the personal jurisdiction of such courts for that purpose.

2) Trademark and Product Attribution Statements.

Qualcomm is a trademark or registered trademark of Qualcomm Incorporated. Arm is a registered trademark of Arm Limited (or its subsidiaries) in the U.S. and/or elsewhere. The Bluetooth® word mark is a registered trademark owned by Bluetooth SIG, Inc. Other product and brand names referenced in this Material may be trademarks or registered trademarks of their respective owners.

Snapdragon and Qualcomm branded products referenced in this Material are products of Qualcomm Technologies, Inc. and/or its subsidiaries. Qualcomm patented technologies are licensed by Qualcomm Incorporated.